

Facts, plans, and beliefs.

Reason beyond prediction.

Explore symbolic and probabilistic AI,
from explicit rules to decisions under uncertainty.

🕒 15-20 min core reading · 20-30 min practice · 12 short sections

“Pick a movie for our offline train journey. We have 100 minutes, and I liked that sci-fi trailer.” Part of this request is a strict rule, part is uncertain preference, and part may require a sequence of actions. AI has tools for each of these, including many that do not require a neural network.

WHAT YOU WILL UNDERSTAND

The main question: how can an AI system represent facts, find a plan, and make a decision with incomplete information?

The practical path: express constraints, search possible steps, update a belief with evidence, and compare the value of feasible choices.

Reading path: twelve short sections, three diagrams, a movie reasoning lab, and a Python workbook. Open the optional notes for the deeper connections.

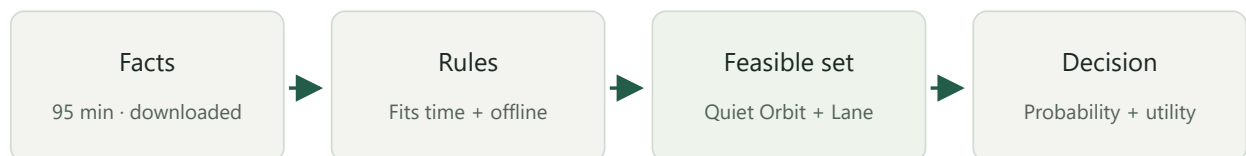
1. AI includes more than learned predictions

Symbolic AI represents knowledge explicitly using objects, facts, rules, and relationships. **Search** explores possible solutions. **Probabilistic AI** represents uncertainty with distributions. These approaches overlap: a planner can use learned estimates, and a rule system can consume a probabilistic prediction. “Classical AI” is a broad historical label, not a precise boundary excluding machine learning.

Movie-night question	Useful representation
Is this film short enough and available offline?	Facts and constraints.
Which actions get us from an unprepared device to a ready movie?	States, actions, and a search graph.
Does liking this trailer suggest a sci-fi mood?	A probability model and evidence.
Which allowed choice is worth making?	Expected utility, including an option to skip.

These are different tasks. A film can be likely to please you and still exceed your available time. A valid plan can be expensive. A logically correct conclusion can fail in practice if its starting facts are wrong.

➤ **Separate what is allowed from what is preferred.**



A high enjoyment score cannot override an offline constraint.

No feasible film → explain the limit. No worthwhile film → consider skipping.

Figure 1. Trusted catalog facts support explicit eligibility checks. Only feasible films enter the decision. A probability estimates enjoyment; utility expresses the value of watching.

2. Represent facts and draw logical conclusions

A **proposition** is a statement that can be true or false, such as “Quiet Orbit is downloaded.” A **predicate** expresses a property or relation, such as `downloaded(movie)` or `short_enough(movie, time_limit)`. A **knowledge base** stores the relevant facts and rules.

```
Fact: downloaded(QuietOrbit)
Fact: short_enough(QuietOrbit, 100)
Rule: downloaded(m) AND short_enough(m, 100)
      IMPLIES eligible_offline(m)
Conclusion: eligible_offline(QuietOrbit)
```

Deduction derives what follows from those premises. The rule does not establish that Quiet Orbit is enjoyable; it only establishes eligibility under our definition. **Forward chaining** starts with known facts and repeatedly applies matching rules. **Backward chaining** starts with a query, such as `eligible_offline(QuietOrbit)`, then looks for rules and facts that could prove it. See Poole and Mackworth's definite-clause treatment.

An **expert system** combines domain knowledge, an inference procedure, and often an explanation of which rules fired. It can be useful when conditions are explicit and stable. Maintaining rules, resolving conflicts, and handling missing facts can become difficult as the domain grows.

▼ Go deeper: truth, missing knowledge, and proof guarantees

"We cannot prove downloaded" need not mean "we proved not downloaded." Under an **open-world assumption**, absent facts remain unknown. Under a **closed-world assumption**, an application may treat unrecorded facts as false when its data is intended to be complete. Our offline catalog uses the latter convention with explicit Boolean fields. That is a modeling choice, not a law of logic.

A **sound** inference procedure derives only consequences of its premises. A **complete** one can derive every consequence in the logic under discussion. Neither guarantees that the premises describe reality. First-order logic adds variables and quantifiers: "every downloaded short film is eligible" uses "for all"; "some film fits" uses "there exists." Richer languages need more expensive reasoning. Restricted rule languages such as `Datalog` trade expressive freedom for useful structure.

3. Search through possible states

A search problem specifies an **initial state**, available **actions**, a **transition model**, a **goal test**, and usually an **action cost**. A state is the information needed to decide what can happen next. A path is a sequence of transitions. The **frontier** holds paths waiting to be explored; the search strategy decides which to examine next.

Our small movie-preparation graph has two routes to Ready. A bundled preparation action costs 10 fictional effort units. Checking, choosing, and preparing separately costs $1 + 1 + 1$. A detour returns to Start. All routes and costs are authored teaching data.

➤ Fewest actions does not mean lowest cost.

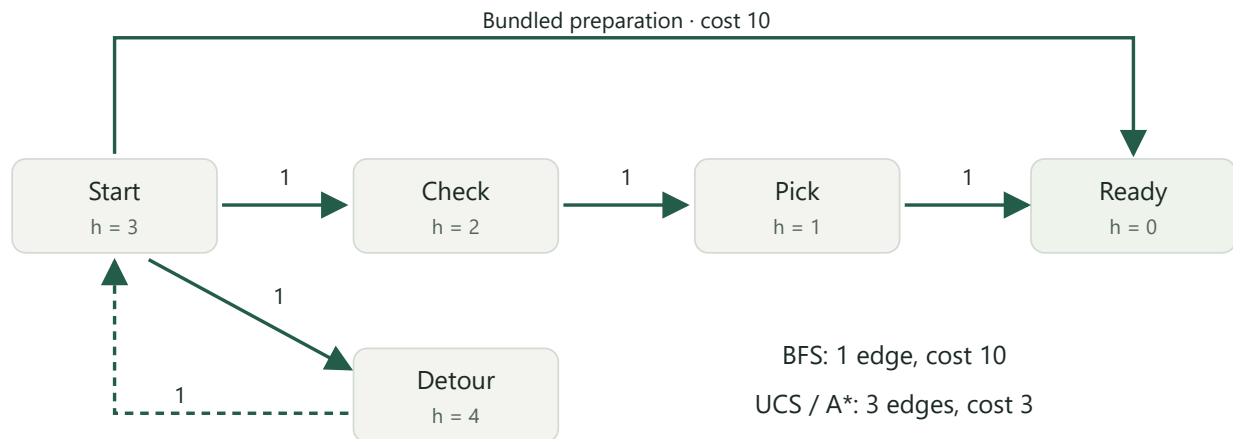


Figure 2. The direct edge costs 10. The three-step route costs 3. A detour cycles back to Start. Numbers beside edges are costs; h is estimated remaining cost. Every arrow is directed.

Strategy	Which path is next?	Main tradeoff
Breadth-first search (BFS)	Fewest edges from the start.	Finds a shallowest goal; cheapest only when step costs are equal.
Depth-first search (DFS)	Follow one branch deeply, then backtrack.	Often uses less frontier memory; may miss cheaper routes or follow an endless branch.
Uniform-cost search (UCS)	Lowest accumulated cost g .	Finds a cheapest solution under standard cost conditions; may explore many paths.
Greedy best-first	Lowest estimate h .	Can reach a goal quickly, but ignores the cost already paid.
A*	Lowest $f = g + h$.	Combines actual cost with an estimate of what remains.

BFS returns Start → Ready, cost **10**. UCS returns Start → Check → Pick → Ready, cost **3**. A* returns the same cheaper route using the heuristic below. Search over possible states differs from chapter 16’s search for relevant documents. The [uninformed search discussion](#) covers BFS, DFS, and UCS.

4. Understand heuristics and search guarantees

A **heuristic** estimates remaining cost. In our graph, use $h(\text{Start}) = 3$, $h(\text{Check}) = 2$, $h(\text{Pick}) = 1$, $h(\text{Detour}) = 4$, and $h(\text{Ready}) = 0$. After Start is expanded, the direct goal has $f = 10$, Check has $f = 1 + 2 = 3$, and Detour has $f = 1 + 4 = 5$. A* explores Check first and reaches Ready through the cheaper route.

An **admissible** heuristic never overestimates the true cheapest remaining cost. A **consistent** heuristic also satisfies $h(n) \leq c(n, n') + h(n')$ along every edge, with zero at a goal. These properties matter to A*’s optimality guarantees. Setting $h = 0$ reduces A* to UCS. [Informed search](#) explains the conditions.

Our workbook records a best cost per state, allows a cheaper path to replace an earlier one, and tests the goal when it is removed from the frontier. Returning when Ready is first *generated* would wrongly select cost 10. The supplied heuristic is consistent; on this graph A* avoids the detour that UCS examines.

▼ Go deeper: completeness, cycles, memory, and adversarial search

For a finite graph, tracking visited states prevents repeatedly traversing a cycle. With infinitely many states, the conditions are more delicate: BFS needs finite branching and a goal at finite depth; UCS and A* commonly assume finite branching and step costs bounded below by a positive value. A* graph search must handle improved paths correctly; an admissible but inconsistent heuristic can require reopening states. A learned heuristic can guide search without satisfying these guarantees. See [search pruning](#).

Large frontiers can exhaust memory even when an algorithm is correct. Beam search keeps only a limited number of candidates and can discard the best route. In two-player zero-sum games, **minimax** models alternating choices by a maximizing player and a minimizing opponent; **alpha-beta pruning** skips branches that cannot change the minimax decision. A movie planner normally has no adversarial opponent, so ordinary path search is the closer match.

5. Planning adds the meaning of actions

A planner searches using an action model. Each action has **preconditions** that must hold and **effects** describing the state afterward. In a simple deterministic model, `download(movie)` requires connectivity and enough storage; its effect is that the movie becomes available offline. `watch(movie)` requires availability and sufficient time; it consumes time and records that it was watched.

```
Initial: online, enough_storage, not_downloaded
Action:  download(QuietOrbit)
Result:  online, enough_storage, downloaded
Action:  go_offline
Goal:    offline AND downloaded
```

An action description should preserve unaffected facts and explicitly add or remove changed ones. Our sketch omits download duration and exact storage use; a real planner would need them. If the device is already offline, the download precondition fails. Generating “download, then watch” as fluent text does not establish that the plan is executable.

Forward planning searches from the known initial state toward a goal. A classical deterministic planner commonly assumes a known state and predictable action effects. Real tools can fail or change state, so execution needs observation and replanning. This is the bridge to chapters 31-32’s workflows and agents. See [forward planning](#).

6. Constraints describe what a solution must satisfy

A **constraint satisfaction problem (CSP)** consists of variables, possible values called **domains**, and constraints on valid combinations. For a two-film evening, variables are `first_movie` and `second_movie`; domains contain the catalog; constraints might require different films, both downloaded, and a combined duration of at most 190 minutes.

Backtracking assigns values and returns to an earlier choice when a constraint fails. **Constraint propagation** removes values that cannot work with the remaining possibilities. Neither means the system has learned a preference. Once feasible assignments exist, **optimization** chooses the best according to a separate objective. See [CSP search](#) and [propagation](#).

In the lab, 100 minutes and offline access leave Quiet Orbit (95 minutes) and Lantern Lane (88 minutes). Comet Chase is both too long and not downloaded; Silent Harbor is too long. With 80 minutes, **no movie is feasible**. The correct response is to explain that, or ask which constraint may change. Quietly replacing 80 with 100 breaks the stated problem.

▼ Go deeper: hard rules, soft preferences, SAT, and optimization

A **hard constraint** must hold. A **soft preference** affects a score but may be traded against other preferences. “Offline only” is hard here; “probably enjoyable” is soft. A large preference score cannot cancel an offline violation. Filter or constrain first, then optimize within the feasible set.

SAT asks whether Boolean variables can make a logical formula true. **SMT** adds supported theories such as arithmetic. Integer programming represents decisions with integer variables and algebraic constraints, often optimizing a numeric objective. These are related ways to formalize problems, with different modeling and solver strengths. Our four-row enumeration illustrates the distinction between feasibility and value; it is not an implementation of a general CSP, SAT, or SMT solver.

7. Update uncertain beliefs with Bayes’ rule

Let H mean “the viewer is in a sci-fi mood tonight” and E mean “the viewer liked a sci-fi trailer.” A **prior** is the belief before this evidence. A **likelihood** describes how probable the evidence is under a hypothesis. A **posterior** is the updated belief after observing it. Revisit chapter 7 if conditional probability feels unfamiliar.

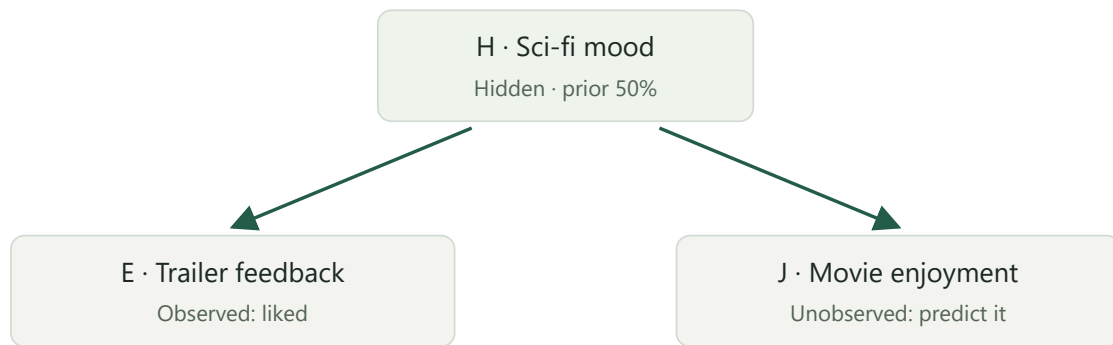
Our fictional model sets $P(H) = 0.5$, $P(E \mid H) = 0.8$, and $P(E \mid \neg H) = 0.2$. Bayes’ rule gives:

$$P(H \mid E) = \frac{P(E \mid H)P(H)}{P(E \mid H)P(H) + P(E \mid \neg H)P(\neg H)} = \frac{0.8 \times 0.5}{0.8 \times 0.5 + 0.2 \times 0.5} = 0.8.$$

The denominator counts both ways the evidence could occur. A liked trailer raises our belief to **80%**, not certainty. Unknown trailer feedback leaves the prior unchanged. Disliked feedback uses complementary likelihoods: 0.2 under H and 0.8 otherwise. Missing feedback is not the same as disliked feedback. See the [probability foundations](#).

A **Bayesian network** organizes random variables in a directed acyclic graph, with a probability distribution for each variable given its parents. The graph and these distributions define a joint distribution; the arrows encode a chosen dependency structure. See [belief networks](#).

➤ Evidence updates a hidden belief, then a prediction.



Liked trailer $\rightarrow P(H | E) = 80\% \rightarrow P(\text{enjoy Quiet Orbit} | E) = 78\%$

Figure 3. For a fixed movie, the model factors as $P(H) \times P(E | H) \times P(J | H)$. E is trailer feedback, H is mood, and J is enjoyment. The update runs from observed feedback to the hidden mood; arrows specify the probability model, not proof of causation.

For a fixed movie, our model assumes trailer feedback and enjoyment are conditionally independent given the mood. Quiet Orbit has authored enjoyment probabilities 0.9 in a sci-fi mood and 0.3 otherwise. After the liked trailer:

$$P(\text{enjoy Quiet Orbit} | E) = 0.8(0.9) + 0.2(0.3) = 0.78.$$

The **80% mood probability** and **78% enjoyment probability** answer different questions.

▼ Go deeper: inference, independence, and causal claims

Marginalization sums over unobserved possibilities, as we just did over the two moods. Exact inference methods include enumeration and variable elimination; sampling approximates quantities when exact calculation is too expensive. Efficiency depends on network structure as well as variable count. A hidden Markov model extends probabilistic reasoning to a sequence with hidden states and observations. See the [uncertainty review](#).

Do not multiply likelihoods for repeated evidence unless the required conditional independence is justified. Counting the same trailer feedback twice would exaggerate certainty. The probabilities here are authored, not estimated from users or calibrated on held-out data. A real model needs evidence for its numbers and assumptions. A Bayesian-network arrow does not by itself prove causation; a causal interpretation needs additional assumptions. Chapter 40 develops that distinction.

8. Choose actions using expected utility

Probability describes possible outcomes; **utility** expresses how desirable those outcomes are for a particular decision. **Expected utility** weights each outcome's utility by its probability. Choose among feasible actions, including doing nothing when appropriate. The [preferences and utility](#) chapter develops this decision-theoretic view.

Our lab assigns enjoyment **+5**, disappointment **-4**, and skipping **0**. These are fictional preference units, not money or universal human values. If enjoyment probability is p :

$$EU(\text{watch}) = 5p - 4(1 - p) = 9p - 4.$$

Quiet Orbit has $p = 0.78$, so its expected utility is **3.02**. Lantern Lane has $p = 0.48$, giving **0.32**. Both satisfy the default constraints, so choose Quiet Orbit. Watching beats skipping only when $p > 4/9$. If disappointment instead costs 10, that threshold becomes **10/15**: beliefs can stay the same while the preferred action changes.

▼ Go deeper: combine learned perception with explicit reasoning

A **hybrid** system can use a neural model to interpret "a short, quiet film," a structured catalog to check runtime, a solver to enforce constraints, and a probabilistic model to rank feasible choices. **Neuro-symbolic AI** explores combinations of learned representations and symbolic structure, with many possible designs rather than one standard architecture.

Validate the boundaries: an LLM-extracted duration is a proposal until grounded in trusted data. Symbolic reasoning cannot repair a wrong premise automatically. Explicit rules are inspectable but costly to maintain; learned systems generalize but can make opaque errors. Test each component and the full decision. In our lab, no text model, learning algorithm, or general theorem prover is running: fixed code applies the authored rules and probability tables.

9. Run the Python reasoning workbook

Extract the ZIP and open a terminal in that folder. Use Python 3.10 or later. No packages or API keys are needed.

```
python reasoning.py
python reasoning.py --minutes 120 --offline no
python reasoning.py --evidence unknown --penalty 10
python reasoning.py --minutes 80
python -m unittest -v test_reasoning.py
```

File	What to inspect
data.json	Four fictional films, probability tables, graph edges, and heuristic.
reasoning.py	Eligibility checks, Bayes update, expected utility, BFS, UCS, and A*.
test_reasoning.py	Hand calculations, constraints, abstention, all settings, and search guarantees on this graph.
README.md	Commands, assumptions, limitations, and suggested experiments.

The default JSON reports `posterior: 0.8` , `eligible: 2` , `chosen: "M01"` , and `bestUtility: 3.02` . The separate `searches` field reports BFS cost 10 and UCS/A* cost 3. Changing movie preferences does not change this fixed preparation graph.

The workbook uses a sorted list for search clarity and enumerates just four movies. It rounds report probabilities and utilities to six decimal places; selection uses those utilities. Ties prefer skipping at zero, then the earlier eligible catalog film. This is a readable teaching implementation, not a scalable solver or a trained recommender.

10. Experiment with the movie reasoning lab

Rules first. Then reason under uncertainty.

Worked decision: 100 minutes, offline required, prior 50%, liked trailer, disappointment cost 4. The posterior is 80%; only the first two films are eligible.

Movie	P(enjoy)	Expected utility	Allowed?
Quiet Orbit	78%	3.02	Yes · chosen
Lantern Lane	48%	0.32	Yes
Comet Chase	86.4%	3.776	No · time + offline
Silent Harbor	58%	1.22	No · time

At 120 minutes with online access, Comet Chase wins. At default time with unknown feedback and penalty 10, both eligible utilities are -1 : skip. At 80 minutes, no film is feasible. Separate graph: BFS cost 10; UCS and A* cost 3.

Exercise 1: a strong score cannot override a rule

Set 120 minutes, liked feedback, a 50% prior, and disappointment cost 4. Compare offline required with online allowed. Which choice changes, and why?

▼ Reveal the explanation

Offline, Quiet Orbit wins. Online, Comet Chase becomes feasible and wins with enjoyment probability 0.864 and expected utility 3.776. Its score was already high while offline; the missing download prevented selection.

Exercise 2: distinguish uncertainty from a negative observation

Use default 100 minutes and offline mode, a 50% prior, unknown feedback, and disappointment cost 10. Then change only the feedback to liked.

▼ Reveal the explanation

With unknown feedback, the posterior remains 0.5 and both eligible films have enjoyment probability 0.6. Their utility is -1 , so skip. Liked feedback raises the posterior to 0.8; Quiet Orbit then has utility 1.7 and wins. At an 80-minute limit there would instead be no feasible movie, regardless of the belief.

Exercise 3: choose the right search objective

Inspect the search traces. Why does BFS return cost 10 while UCS and A* return cost 3? Why should UCS not stop when it first sees Ready?

▼ Reveal the explanation

BFS minimizes the number of edges, so the single direct edge wins. UCS minimizes accumulated cost and A* uses cost plus a valid remaining-cost estimate. Ready is initially discovered through the expensive edge; the cheaper path reaches it later. Testing the goal when it is popped preserves the intended cheapest-path behavior here.

11. Check your understanding

01 A film has the highest expected utility but is not downloaded. The user requires offline playback. What should happen?

- A. Exclude it from the feasible choices.
- B. Recommend it because the score is highest.
- C. Silently assume internet access.

Answer: A. A soft preference score does not override a hard constraint. Explain infeasibility if no allowed film remains.

02 One route has one edge costing 10; another has three edges costing 1 each. What does BFS optimize?

- A. Total cost regardless of edge weights.
- B. Number of edges, so it can return the cost-10 route.
- C. Expected movie enjoyment.

Answer: B. BFS returns a shallowest goal. Equal edge costs are needed to equate fewest edges with lowest cost.

03 What does an admissible A* heuristic mean?

- A. It always equals the true remaining cost.
- B. It may overestimate whenever a model is confident.
- C. It never overestimates the cheapest remaining cost.

Answer: C. Admissibility is a mathematical bound. A* graph search must also handle repeated states and improved paths correctly; consistency is a stronger useful condition.

04 No trailer feedback was recorded. How does the teaching model update the mood probability?

- A. Treat it as disliked feedback.
- B. Keep the prior unchanged.
- C. Set the probability to zero.

Answer: B. Missing evidence is not a negative observation. The lab only changes the belief for observed liked or disliked feedback.

05 Two films satisfy all constraints, but each has expected utility –1 and skipping has utility 0. Which conclusion is correct?

- A. The constraint problem has no solution.
- B. A movie must be chosen because one is feasible.
- C. Skipping has greater expected utility under these assumptions.

Answer: C. Feasibility and desirability are different. Abstention can be preferred even when valid options exist.

12. Revise and connect the ideas

Try explaining the movie decision without opening the notes: **facts support rules; rules restrict choices; search explores paths; probability updates beliefs; utility selects an action.** Name one assumption that could make each step wrong in a real application.

▼ Quick-reference glossary

Term	Meaning
Knowledge base / inference	Stored facts and rules / deriving consequences from them.
Predicate / quantifier	A property or relation / “for all” or “there exists.”
State / action / goal	Current modeled situation / allowed transition / desired condition.
Frontier / heuristic	Pending search candidates / estimated remaining cost.
Admissible / consistent	Never overestimates / also respects each edge’s cost inequality.
Planning	Finding actions whose preconditions and effects achieve a goal.
CSP / feasible solution	Variables, domains, and constraints / an assignment satisfying them.
Prior / likelihood / posterior	Belief before / evidence probability under a hypothesis / belief after.
Bayesian network	A directed acyclic probability model with conditional distributions.
Marginalization	Summing or integrating over unobserved possibilities.
Expected utility	Probability-weighted value of an action’s outcomes.
Hybrid / neuro-symbolic	Combining approaches / combining neural and symbolic methods.

The linked sections of Poole and Mackworth’s *Artificial Intelligence: Foundations of Computational Agents* provide the formal foundations. The movie data, diagrams, calculations, and workbook here are original teaching examples. For additional breadth, read their [adversarial search](#) and [constraint reasoning](#) chapters.

Next: Chapter 38 - Reinforcement learning and robotics. We will move from choosing with a supplied model to learning actions through interaction, feedback, and delayed rewards.